REAL-TIME FRAUD MITIGATION IN UPI GATEWAYS USING HYBRID MACHINE LEARNING

PROJECT REPORT – PHASE I

Submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in Computer Science and Engineering

By

[STUDENT NAME 1] (Reg. No – 4361xxxx)

[STUDENT NAME 2] (Reg. No – 4361xxxx)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

CATEGORY - 1 UNIVERSITY BY UGC

Accredited “A++” by NAAC | Approved by AICTE

JEPPIAAR NAGAR, RAJIV GANDHI SALAI, CHENNAI - 600119

AUGUST - 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of [Student Name 1] (Reg. No. 4361xxxx) and [Student Name 2] (Reg. No. 4361xxxx) who carried out the Project Phase-I entitled “Real-Time Fraud Mitigation In Upi Gateways Using Hybrid Machine Learning” under our supervision from June 2026 to August 2026.

Internal Guide

[GUIDE NAME], M.E., Ph.D.,

Head of the Department

Dr. Senduru Srinivasulu, M.Tech., Ph.D.,

Submitted for Project Phase I Examination held on _____

Internal Examiner **External Examiner**

DECLARATION

We, [Student Name 1] (Reg. No. 4361xxxx) and [Student Name 2] (Reg. No. 4361xxxx), hereby declare that the Project Phase-I Report entitled “Real-Time Fraud Mitigation In Upi Gateways Using Hybrid Machine Learning” done by us under the guidance of [Guide Name], M.E., Ph.D., is submitted in partial fulfillment of the requirements for the award of the Bachelor of Engineering degree in Computer Science and Engineering. The work reported herein is original and has not been submitted to any other institution for the award of any degree or diploma.

DATE:

PLACE: Chennai **SIGNATURE OF THE CANDIDATES**

ACKNOWLEDGEMENT

We are pleased to acknowledge our sincere thanks to the Board of Management of Sathyabama Institute of Science and Technology for their kind encouragement in doing this project and for completing it successfully. We are grateful to them.

We convey our thanks to Dr. L. Lakshmanan, M.E., Ph.D., Dean, School of Computing, and Dr. Senduru Srinivasulu, M.Tech., Ph.D., Head of the Department, Department of Computer Science and Engineering, for providing us the necessary support and details at the right time during the progressive reviews.

We would like to express our sincere and deep sense of gratitude to our Project Guide [Guide Name], M.E., Ph.D., for the valuable guidance, suggestions and constant encouragement which paved the way for the successful completion of this project work.

We wish to express our thanks to all Teaching and Non-teaching staff members of the Department of Computer Science and Engineering who were helpful in many ways for the completion of the project.

ABSTRACT

The Unified Payments Interface settles transactions irrevocably within seconds, making recovery of fraudulent transfers largely infeasible since stolen funds are withdrawn through mule accounts before retrospective analysis raises an alert. Conventional analytics operate after settlement and cannot prevent loss on such a rail. This project proposes RT-IIPF, which relocates the fraud decision inside the payment transaction itself. The switch authenticates the customer, locks both ledger rows, applies the debit and credit provisionally without committing, and suspends the executing thread while an isolated microservice scores the transaction. A hybrid engine fuses a deterministic rule tier with two complementary unsupervised detectors, an Isolation Forest and a Local Outlier Factor model, requiring no labelled fraud data. Six user-entered fields are enriched into a thirteen-dimensional vector using client telemetry and server-resolved attributes a hostile client cannot forge. A clean verdict commits the ledger, whereas a fraudulent verdict writes the account lock and audit record in an independent transaction before triggering an atomic rollback, so a blocked transfer never attains durable existence. Evaluated on 250,000 UPI transactions using synthetic fraud injection, the hybrid achieved 0.745 recall at a 0.86 percent false-positive rate, a fivefold improvement over a single Isolation Forest, with the inline decision measured at eleven to twelve milliseconds.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
1	INTRODUCTION	1
	1.1 Overview of UPI and the Fraud Problem	1
	1.2 Motivation	2
	1.3 Problem Statement	3
	1.4 Objectives of the Project	3
	1.5 Scope of the Project	4
	1.6 Organization of the Report	4
2	LITERATURE SURVEY	5
	2.1 Inferences from Literature Survey	10
	2.2 Limitations and Research Gaps in Existing System	11
3	REQUIREMENTS ANALYSIS	13
	3.1 Feasibility Study and Risk Assessment	13
	3.2 Software Requirements and Specifications	17
4	DESCRIPTION OF PROPOSED SYSTEM	20
	4.1 Selected Methodologies	20
	4.2 Architecture Diagram	23
	4.3 Module Description and Workflow	24
	4.4 Estimated Cost for Implementation and Overheads	28
	REFERENCES	29

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1	System Architecture of the Proposed RT-IIPF Framework	23
4.2	Hybrid Scoring Engine – Three-Tier Fusion	22
4.3	Anomaly Score Distribution – Clean Traffic vs Injected Fraud	27
4.4	Detector Comparison at Matched False-Positive Budget	27

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of Literature Surveyed	9
3.1	Risk Assessment and Mitigation Strategy	16
3.2	Hardware and Software Requirements	17
4.1	Thirteen-Dimensional Feature Vector Composition	21
4.2	Detector Performance at Matched False-Positive Rate	26
4.3	Measured System Latency	27
4.4	Estimated Cost and Implementation Overheads	28

CHAPTER 1

INTRODUCTION

1.1 Overview of UPI and the Fraud Problem

The Unified Payments Interface (UPI), introduced by the National Payments Corporation of India, has become the dominant retail payment mechanism in the country. It allows a customer to transfer funds instantly between bank accounts using a Virtual Payment Address (VPA) rather than an account number and IFSC code, authorised by a Mobile Personal Identification Number (MPIN). The system settles transactions in a matter of seconds and operates continuously, which has made it central to everyday commerce for hundreds of millions of users.

The same characteristics that make UPI convenient also make it attractive to fraudsters. Because settlement is immediate and irrevocable, a fraudulent transfer is complete before any human or automated review can intervene. Funds are typically routed through a chain of intermediary or mule accounts and withdrawn within minutes. Common attack patterns include social engineering, in which a victim is persuaded to authorise a payment; credential compromise, in which an attacker obtains the MPIN; malicious collect requests; and fraudulent QR codes that redirect payment to an attacker-controlled address.

Conventional fraud analytics in banking operate retrospectively. Transaction records are written to logs, consumed by batch or streaming pipelines, and scored after the transaction has already committed. Cases that the model flags enter a manual review queue or a recovery workflow. On a traditional card rail, where settlement may take a day or more, this approach is workable because a suspicious transaction can be reversed before funds are released. On a real-time rail such as UPI it is structurally inadequate: detection accuracy achieved after settlement contributes little when recovery is practically infeasible.

1.2 Motivation

The motivation for this project arises from the observation that the fraud decision and the money movement occur at the wrong points relative to each other. If the decision could be relocated so that it occurs while the funds are still under the control of the payment switch, a fraudulent transfer would never attain durable existence in the ledger and there would be nothing to recover. Prevention would replace recovery.

This relocation is conceptually simple but raises genuine engineering difficulties. Holding a payment in flight means holding database resources; if the scoring step is slow, customers experience delay and the system's throughput collapses. If the scoring service becomes unavailable, the system must decide whether to admit unscored payments or reject them. If a transfer is provisionally applied and then reversed, the institution must still retain a durable record of the attempt even though the movement itself has been erased. These difficulties, rather than the choice of classifier, are the substantive obstacles to inline fraud prevention, and they motivate the design presented in this report.

A second motivation concerns the detection technique itself. Supervised learning, which dominates published work on payment fraud, requires labelled examples of fraud. Such labels are scarce, arrive weeks after the event, and describe only fraud that has already been recognised. A model trained on historical labels is therefore blind to novel attack patterns. Unsupervised anomaly detection requires no labels and can flag behaviour it has never seen, which is a decisive advantage in a domain where attackers adapt continuously.

1.3 Problem Statement

To design, implement and evaluate a payment framework in which the fraud decision is taken inside the payment transaction boundary, such that a transaction judged fraudulent is atomically rolled back and never durably recorded in the ledger, while the corresponding security response is durably retained; and to do so within a latency budget small enough that the intervention is imperceptible to a legitimate customer on a real-time payment rail.

1.4 Objectives of the Project

- To design an inline interception pipeline that suspends a UPI payment mid-transaction and arbitrates its commit on a machine learning verdict.
- To implement atomic rollback with a surviving audit trail, so that a blocked transfer leaves no ledger effect while the account lock and audit record persist.
- To develop a hybrid scoring engine that fuses a deterministic rule tier with two complementary unsupervised detectors, requiring no labelled fraud data.
- To enrich six user-entered fields into a thirteen-dimensional feature vector using client telemetry and server-resolved attributes that cannot be forged by a hostile client.

- To establish an evaluation methodology appropriate to a dataset whose fraud labels are unreliable, and to quantify detection performance against it.
- To validate the sub-fifty millisecond design budget through latency and concurrency measurement of the complete multi-service system.

1.5 Scope of the Project

The scope of Phase I covers the design and implementation of a working three-service prototype comprising a client application, an enterprise payment switch with a relational ledger, and an isolated machine learning inference service; the training and calibration of the hybrid scoring engine on a public dataset of two hundred and fifty thousand UPI transactions; and the experimental evaluation of detection performance, latency and concurrency behaviour. The prototype is intended to demonstrate the feasibility of the inline interception mechanism. It is not a production banking system: it uses an in-memory database, prototype-grade cryptography and unencrypted local transport, and it does not integrate with the NPCI network. These boundaries are stated explicitly in Chapter 3 and revisited as future work.

1.6 Organization of the Report

Chapter 2 surveys the existing literature on payment fraud detection and unsupervised anomaly detection, and identifies the research gaps this project addresses. Chapter 3 presents the requirements analysis, comprising the technical, economic and operational feasibility study, a risk assessment with mitigation strategies, and the software requirements and specifications. Chapter 4 describes the proposed system in detail, including the selected methodology, the system architecture, the module-level design and workflow, the experimental results obtained, and the estimated implementation cost. The report concludes with the list of references.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews the body of work relevant to the project in three groups: the foundational unsupervised anomaly detection algorithms on which the proposed scoring engine is built, the evaluation methodology literature that informs how such systems should be measured, and recent applied work on fraud detection for UPI and card payment systems.

Foundational Anomaly Detection Algorithms

Liu, Ting and Zhou (2008) introduced the Isolation Forest, which departs from the prevailing practice of profiling normal data and instead isolates anomalies directly. The method constructs an ensemble of trees by recursively partitioning the data along randomly chosen features at randomly chosen split points. Anomalous points, being few and different, require fewer partitions to isolate, and the average path length across the ensemble yields the anomaly score. The technique has linear time complexity and low memory requirements, which makes it attractive for latency-constrained inference. Its known weakness, confirmed experimentally in this project, is degradation in sparse high-dimensional spaces such as those produced by one-hot encoding of categorical attributes, where axis-parallel splits dilute the informative numeric dimensions.

Breunig, Kriegel, Ng and Sander (2000) proposed the Local Outlier Factor, which assigns each point a degree of outlierness by comparing its local reachability density with that of its neighbours. Because the comparison is local rather than global, the method detects points that lie in a locally sparse region even when they are unremarkable with respect to the overall distribution. This property is complementary to the isolation-based approach and is the reason both detectors are retained in the proposed hybrid engine.

Schölkopf, Platt, Shawe-Taylor, Smola and Williamson (2001) developed the One-Class Support Vector Machine, which estimates a support region enclosing the normal class in a kernel-induced feature space. The formulation is principled and the kernel choice permits non-linear boundaries, but training cost grows quadratically with sample size and performance is sensitive to the kernel and the parameter

controlling the fraction of outliers, which limits its practicality at the data volumes considered here.

Hariri, Kind and Brunner (2021) proposed the Extended Isolation Forest, which replaces axis-parallel splits with randomly oriented hyperplanes in order to remove the artefacts that arise from axis-aligned partitioning. The work is directly relevant to the weakness of the standard Isolation Forest observed in this project and is identified as a candidate for future evaluation.

Evaluation Methodology

Campos, Zimek, Sander and colleagues (2016) conducted a systematic study of the evaluation of unsupervised outlier detection, examining measures, datasets and experimental practice. They demonstrate that the relative ranking of detectors is strongly dependent on the dataset and on the evaluation measure chosen, and they caution against drawing general conclusions from a single benchmark. Their work motivates the comparative evaluation of multiple detectors on a common footing that is reported in Chapter 4, and the use of an operating-point metric rather than a threshold-independent summary alone.

Surveys of Financial Fraud Detection

Abdallah, Maarof and Zainal (2016) surveyed fraud detection systems across credit card, telecommunication and insurance domains, presenting a taxonomy of fraud types and of the detection techniques applied to them. They identify extreme class imbalance, the delayed availability of labels and concept drift as the recurring obstacles in the field. The survey predates the emergence of real-time retail payment rails and does not address the integration of detection with transaction processing.

Adewumi and Akinyelu (2017) reviewed machine learning and nature-inspired techniques for credit card fraud detection, comparing algorithm families and the accuracy ranges reported for each. Their treatment is card-centric and assumes a settlement model in which reversal remains possible after the transaction, an assumption that does not hold on UPI.

Recent Applied Work on UPI Fraud Detection

Deepak Kumar, Harshini Abirami and Hepsi Nambika (2026) proposed a smart UPI fraud detection system that scores transactions with an unsupervised Isolation Forest and, for transactions judged safe, applies a One-Time Password verification step before executing the transfer. The system is implemented as a Flask application and

evaluated on six hundred records drawn from a public dataset. This work is the closest to the present project in both algorithm and domain, and it does maintain a temporary halt on the transaction pending OTP verification. However, the halt is implemented in application control flow rather than within a database transaction: the paper contains no treatment of commit or rollback semantics, of concurrency control, or of atomicity across the multi-step transfer, and reports no latency measurement despite describing the system as operating in real time.

Thayanidhi, Srivarshan and Shalini (2026) applied decision trees, random forests and neural networks to UPI transaction patterns and user behaviour, reporting real-time analysis of behavioural signals. The approach is supervised and therefore depends on the availability of labelled fraud examples, and detection is performed on the transaction record after the fact rather than within the payment path.

Jani, Kumar, Gumasthi, Singla and Kailasam (2025) applied actor-critic reinforcement learning to the UPI fraud decision, allowing the detection policy to adapt as fraud strategies evolve without full retraining. The adaptivity is valuable, but the approach introduces training complexity and sensitivity to reward design, and it provides no mechanism for enforcing the resulting decision at the ledger level.

Kamble, Pisal, Vaidya and Gaikwad (2025) investigated stacked generalisation for UPI fraud detection, combining the outputs of several base learners through a meta-learner. The ensemble improves reported accuracy over individual classifiers and demonstrates the value of combining complementary models, a principle adopted in the hybrid engine proposed here, although their combination is entirely supervised.

Sindhu and Swarupa (2024) and Kavitha, Indira, Shrinita and Bappan (2024) independently evaluated conventional machine learning algorithms on UPI transaction datasets, reporting comparative accuracy figures for logistic regression, decision tree, random forest and support vector approaches. Both studies confirm that tree-based ensembles perform well on tabular payment data, and both evaluate detection in isolation from the payment system.

Chandini, Mahender and Venkateshwarlu (2025) examined the detection of fraudulent UPI transactions through machine learning with emphasis on feature engineering from transaction metadata, and Khopade and Vitalkar (2025) reported a comparable study, both concluding that metadata such as device, frequency and

location materially improves detection. This finding directly informs the feature enrichment strategy described in Chapter 4.

Fatlawi (2025) combined an Isolation Forest with multi-cluster deep learning for fraud detection, reporting improved separation over the isolation method used alone. The work supports the central premise of the present project's scoring design, namely that no single unsupervised detector is sufficient and that combination yields better coverage than any constituent.

Table 2.1: Summary of Literature Surveyed

Author (Year)	Technique	Limitation Relevant to This Work
Liu et al. (2008)	Isolation Forest	Degrades on sparse one-hot categorical spaces; scores saturate beyond the training range
Breunig et al. (2000)	Local Outlier Factor	Requires a resident reference set; no transaction-level integration
Schölkopf et al. (2001)	One-Class SVM	Quadratic training cost; sensitive to kernel and nu selection
Hariri et al. (2021)	Extended Isolation Forest	Addresses axis-parallel artefacts but not deployment or latency
Campos et al. (2016)	Evaluation study	Establishes measurement practice; does not address operating-point constraints
Abdallah et al. (2016)	Survey of fraud detection	Predates real-time rails; no transactional enforcement
Adewumi & Akinyelu (2017)	Survey of ML techniques	Card-centric; assumes post-settlement reversal is possible
Deepak Kumar et al. (2026)	Isolation Forest + OTP	Halt is in application flow, not a database transaction; no rollback; no latency measured
Thayanidhi et al. (2026)	Decision tree / RF / NN	Supervised; requires labelled fraud; scoring is post-transaction
Jani et al. (2025)	Actor-critic RL	Training complexity; no ledger-level enforcement
Kamble et al. (2025)	Stacked generalisation	Entirely supervised; no inline integration
Sindhu & Swarupa (2024)	Comparative ML study	Detection evaluated in isolation from the payment system
Kavitha et al. (2024)	Comparative ML study	No deployment architecture or latency analysis

Chandini et al. (2025)	Metadata-based ML	Confirms value of metadata; no transactional integration
Fatlawi (2025)	Isolation Forest + deep clustering	Supports combination principle; no real-time enforcement

2.1 Inferences from Literature Survey

- Research on UPI fraud detection is overwhelmingly post-transaction: models score a payment after it has settled, leaving recovery as the only available remedy on a rail where recovery is rarely successful.
- Supervised learning dominates the applied literature, despite the acknowledged scarcity, latency and incompleteness of real fraud labels, which limits the ability of such systems to respond to previously unseen attack patterns.
- No reviewed work integrates the fraud verdict with database transaction semantics. None provides atomic rollback of a flagged transfer, concurrency control during scoring, or a guarantee that the security record survives the reversal it describes.
- Reported accuracy is rarely accompanied by latency measurement. Several works describe their systems as operating in real time without quantifying the decision time, so the feasibility of inline deployment remains unestablished.
- Evaluation frequently relies on accuracy or on threshold-independent summaries such as ROC-AUC, both of which are misleading for a blocking system that must operate under a strict false-positive budget on heavily imbalanced data.
- The literature consistently reports that combining complementary models outperforms any single model, which supports the hybrid design adopted in this project, although existing combinations are almost exclusively supervised.

2.2 Limitations and Research Gaps in Existing System

Limitations of the Existing System

- Detection occurs after settlement, so a correct verdict does not prevent financial loss on an irrevocable real-time rail.
- The fraud decision is architecturally separated from the money movement, leaving a window between the check and the transfer during which the account state may change.

- Absence of concurrency control permits a time-of-check-to-time-of-use race, in which two concurrent payments may each pass a balance check and both execute.
- A multi-step transfer performed outside a transaction boundary has no atomicity guarantee, so a failure between the debit and the credit leaves inconsistent state.
- Supervised dependence on labelled fraud renders existing systems blind to novel attack strategies until sufficient labelled examples accumulate.
- Single-model scoring inherits the specific blind spots of the chosen algorithm, with no compensating detector.

Research Gaps Addressed

- Gap 1 – Transactional enforcement. No surveyed system arbitrates an open database transaction on a machine learning verdict. This project holds both ledger rows under pessimistic locks, applies the movement provisionally, and commits or rolls back atomically on the verdict.
- Gap 2 – Durable security response under rollback. Erasing a fraudulent transfer must not erase the record of the attempt. This project commits the account lock and audit row in an independent transaction that survives the rollback.
- Gap 3 – Quantified real-time feasibility. Claims of real-time operation are seldom measured. This project defines a sub-fifty millisecond budget in advance and reports measured end-to-end latency and concurrency behaviour against it.
- Gap 4 – Label-free hybrid detection. Existing combinations are supervised. This project fuses a deterministic rule tier with two complementary unsupervised detectors, requiring no labelled fraud data at any stage.
- Gap 5 – Evaluation under unreliable labels. Where public dataset labels are uninformative, accuracy figures computed against them are meaningless. This project adopts perturbation-based synthetic fraud injection with an explicit negative control.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Feasibility Study and Risk Assessment

3.1.1 Feasibility

The feasibility study evaluates whether the proposed inline fraud mitigation framework can be developed and operated successfully within the constraints of an academic project, and whether the resulting system would be viable in a production banking context. Three dimensions are considered: technical, economic and operational.

3.1.1.1 Technical Feasibility

The proposed system is technically feasible because every capability it requires is provided by mature, widely deployed technology. The transactional guarantees on which the design depends are standard relational database features: pessimistic row locking is expressed through the SQL SELECT ... FOR UPDATE construct, and atomic rollback is the defining property of an ACID transaction. The Spring Framework exposes both declaratively, and its propagation semantics provide precisely the mechanism required for the audit record to survive the rollback that erases the money movement.

The machine learning requirement is equally well supported. Both detectors used in the hybrid engine are available in scikit-learn, are unsupervised and therefore require no labelled fraud corpus, and have inference costs measured in single-digit milliseconds on commodity hardware. The decisive technical question was whether the complete inline path could complete within a latency budget acceptable on a live payment rail. This was answered empirically: the measured end-to-end decision time is eleven to twelve milliseconds, well inside the fifty millisecond budget set at design time, confirming technical feasibility rather than assuming it.

The heterogeneous three-service architecture introduces a network hop between the switch and the scoring service, which was a legitimate concern because ledger locks are held for its duration. Measurement showed this hop to cost nine to fifteen milliseconds including HTTP overhead, and a concurrency study established that the pessimistic locks are not the throughput constraint at the levels examined.

Development requires only a Java Development Kit, a Python interpreter and Node.js, all freely available and already familiar.

3.1.1.2 Economic Feasibility

The project is economically feasible because it is built entirely from open-source components with no licensing cost. Spring Boot, the H2 database, FastAPI, scikit-learn, pandas, React and Vite are all free for academic and commercial use. The transaction dataset used for training is a publicly available corpus and incurs no acquisition cost.

No specialised hardware is required. Both detectors train in under a minute on an ordinary laptop, and inference is performed on the CPU, so neither GPU hardware nor cloud compute is needed. The serialised model bundle occupies approximately sixteen megabytes and is loaded once into memory at service startup, so there is no recurring storage or retrieval cost. The total monetary cost of the Phase-I implementation is therefore effectively zero, as itemised in Table 4.4.

Considered as a prospective production system, the economic argument is stronger still. The cost of preventing a fraudulent transfer is a few milliseconds of compute, whereas the cost of failing to prevent one is the full transaction value plus the operational expense of investigation and customer remediation. Because the scoring tier is stateless, capacity can be increased by replication on commodity hardware rather than by vertical scaling.

3.1.1.3 Operational Feasibility

The system is operationally feasible because it imposes no additional burden on the customer. The fraud check is invisible in the normal case: the user enters the payee address, amount and MPIN exactly as in any existing UPI application, and the eleven to twelve millisecond decision is imperceptible against the several hundred milliseconds of ordinary network and interface latency. No new interaction step is introduced for a legitimate payment.

The device telemetry that contributes to the feature vector is captured automatically by the client at page load and requires no user action. Profile and temporal attributes are resolved by the switch from its own database and clock. Consequently the enrichment that gives the model its discriminating power costs the customer nothing in effort.

For the institution, the system is designed to fail safely. If the scoring service becomes unreachable the switch rejects payments rather than admitting them unscored, so a failure of the intelligence tier cannot become a silent security failure. Every decision, whether a commit or a block, is written to an immutable audit table with its score, verdict and processing time, giving operations staff a complete and queryable record. The one operational consideration requiring policy attention is the customer friction caused by false positives, which is quantified in Chapter 4 and addressed through the graduated-response design discussed there.

3.1.2 Risk Assessment

Although the project is feasible, several risks were identified during design and development. Each is stated below with the mitigation adopted.

Table 3.1: Risk Assessment and Mitigation Strategy

Risk	Impact	Likelihood	Mitigation Adopted
Scoring service unavailable while payments are in flight	High	Medium	Fail-closed policy with bounded connect and read timeouts; payment rejected with HTTP 503 rather than admitted unscored
Deadlock between two concurrent payments touching the same accounts	High	Medium	Row locks always acquired in ascending account-identifier order, making a circular wait impossible
Ledger locks held across the network hop degrade throughput	Medium	High	Tight timeouts; concurrency benchmark performed, which showed the locks are not the limiting factor at the levels tested
False positives block legitimate customers	High	High	Threshold calibrated to bound the false-positive rate below one percent; sensitivity sweep reported; graduated response identified as required for production
Audit record lost when the fraudulent transaction is rolled back	High	Medium	Account lock and audit row committed in an independent REQUIRES_NEW transaction before the exception is raised

Public dataset labels unreliable, invalidating measured accuracy	High	High	Labels tested and found uninformative; evaluation switched to perturbation-based synthetic injection with a negative control
Single detector inherits algorithm-specific blind spots	Medium	High	Hybrid engine fusing a rule tier with two complementary unsupervised detectors; per-tier contribution measured
Client-supplied telemetry may be forged by a hostile client	Medium	Medium	Decisive features resolved server-side from the database and switch clock; client-supplied features are those where forgery increases anomaly
In-memory database loses all state on restart	Low	High	Accepted for the prototype as it provides deterministic test resets; file-backed persistence is a single configuration change

3.2 Software Requirements and Specifications

3.2.1 Software Requirements

The development and execution of the proposed system requires the following hardware and software environment. All software components are open-source and freely available.

Table 3.2: Hardware and Software Requirements

Category	Component	Specification
Hardware	Processor	Multi-core x86-64 or ARM64, 2.0 GHz or higher
	Memory	8 GB RAM minimum (16 GB recommended for training)
	Storage	2 GB free space for tooling, dataset and model bundle
Presentation	React 18 with Vite	Client application and operations monitor
	Node.js 18 or later	Build tooling and development server
Switch	Java Development Kit 21	Runtime for the enterprise switch
	Spring Boot 3.5	Web, Data JPA, Validation starters
	Apache Maven 3.9	Build and dependency management

Database	H2 Database Engine	Embedded relational ledger
Intelligence	Python 3.10 or later	Runtime for the scoring service
	FastAPI with Uvicorn	Asynchronous inference API
	scikit-learn 1.4 or later	Isolation Forest and Local Outlier Factor
	pandas, NumPy, joblib	Data preparation and model serialisation

3.2.2 Software Specifications

The system is specified as three cooperating services communicating over HTTP with JSON payloads. This decomposition permits each service to be developed, deployed, restarted and scaled independently, and it is what makes the transactional separation possible: the switch owns the ledger and all money logic, while the intelligence tier remains stateless.

Functional Specifications

- The client shall accept a payee virtual payment address, amount, transaction type, merchant category code and MPIN, and shall capture device and network telemetry without user interaction.
- The switch shall expose a payment endpoint that authenticates the payer by MPIN digest, rejects locked accounts, and validates the request against declared constraints before any ledger operation.
- The switch shall acquire pessimistic write locks on the payer and payee balance rows in ascending account-identifier order prior to applying any movement.
- The switch shall apply the debit and credit within an open transaction without committing, and shall suspend the executing thread pending the scoring verdict.
- The intelligence service shall accept a thirteen-dimensional feature vector and return a normalised anomaly score, a binary verdict and the per-tier score breakdown that justifies it.
- On a clean verdict the switch shall write a success audit record and commit the transaction; on a fraudulent verdict it shall commit the account lock and blocked audit record in an independent transaction, then roll back the movement atomically.
- The switch shall reject payments with an explicit error when the scoring service is unreachable, and shall never admit an unscored payment.

- The client shall present a distinct success or security-block outcome and shall display a live audit stream of recent transaction decisions.

Non-Functional Specifications

- Latency: the complete inline decision shall complete within fifty milliseconds under single-client conditions.
- Consistency: no partial transfer shall ever be observable; the ledger shall move fully or not at all.
- Durability: the security response to a fraudulent verdict shall persist irrespective of the rollback of the associated movement.
- Safety: unavailability of the intelligence tier shall degrade to rejection, never to admission.
- Auditability: every decision shall be recorded with its score, verdict and processing time.
- Explainability: each verdict shall be accompanied by the contribution of each scoring tier.
- Security: MPINs shall never be stored in plaintext and shall be compared in constant time.

CHAPTER 4

DESCRIPTION OF PROPOSED SYSTEM

4.1 Selected Methodologies

The proposed system, designated RT-IIPF (Real-Time Inline Interception Payment Framework), is built on two methodological choices that together distinguish it from the systems reviewed in Chapter 2: inline transactional interception as the enforcement mechanism, and hybrid unsupervised scoring as the detection mechanism.

4.1.1 Inline Transactional Interception

The central methodological decision is to relocate the fraud verdict inside the database transaction that performs the money movement, rather than placing it before or after that transaction. When a payment request is received, the switch opens a transaction, acquires pessimistic write locks on both ledger rows, and applies the debit and credit to the persistence context without committing. The executing thread is then suspended while the scoring service is consulted. The verdict determines whether that open transaction commits or aborts.

This differs materially from a check performed before the transfer begins. A pre-transfer check leaves a window between the moment of checking and the moment of acting during which the account state may change, permitting a time-of-check-to-time-of-use race in which two concurrent payments each pass the balance check and both execute. Holding the rows under lock for the duration of the decision closes that window. It also confers atomicity across the multi-step transfer, so that a failure between the debit and the credit cannot leave the ledger inconsistent.

A consequence of provisionally applying the movement is that rejecting a transaction requires reversal rather than mere abstention. This is achieved by raising a custom unchecked exception from within the transactional boundary, which causes the framework to mark the transaction rollback-only and discard every uncommitted change atomically. The observable postcondition of a blocked payment is that both balances are exactly as they were before the request arrived.

4.1.2 Durable Security Response Under Rollback

Reversing a fraudulent transfer creates an apparent contradiction: the movement must be erased, yet the institution must retain a permanent record of the attempt and

must immobilise the compromised account. Writing both inside the payment transaction would destroy the record along with the movement.

The methodology resolves this using transaction propagation. The component responsible for the security response is annotated to require a new transaction, which causes the framework to suspend the payment transaction, execute and commit the account lock and the blocked audit record independently, and then return control. Only after that independent commit has completed is the exception raised that rolls back the payment. The security response is therefore durable by construction, not by chance.

4.1.3 Hybrid Unsupervised Scoring

Detection is performed by a hybrid engine that fuses three tiers, chosen because each fails in a manner the others do not. No tier requires labelled fraud data at any stage.

The first tier is a deterministic rule engine encoding explicit domain knowledge: an amount beyond the ninety-ninth-point-nine percentile of observed behaviour, a high-value transfer during the small hours, an amount implausible for the declared merchant category, and a high-value transfer from an uncommon channel. Its thresholds are derived from the training distribution rather than hard-coded, so they transfer to a differently scaled deployment. Its value is threefold: it is fully explainable, it continues to fire on values beyond the training range where isolation-based scoring saturates, and it remains available if the machine learning tiers are degraded.

The second tier is an Isolation Forest of two hundred estimators, which scores a transaction by how few random partitions are required to isolate it. It is inexpensive and identifies globally extreme transactions.

The third tier is a Local Outlier Factor detector with twenty neighbours operating in novelty mode over a forty-thousand-point reference set. It compares the local density around a transaction with the density around its neighbours, and therefore identifies transactions that are unremarkable globally but occupy a locally sparse region. This is precisely the class of anomaly the isolation-based tier systematically misses, and experimental results in Section 4.3.7 confirm the complementarity.

Each machine learning tier produces a raw score which is normalised to the unit interval using minimum and maximum constants derived from the training distribution, with the upper bound winsorised at the ninety-nine-point-nine-five percentile so that a

single extreme training point cannot compress the operational range. The normalised scores are combined by weighted fusion, with weights of 0.20 for the isolation tier and 0.80 for the density tier selected by a sweep on the evaluation set. The rule risk then acts as a floor on the fused score, so a transaction the domain rules consider risky cannot be scored below that level by the models, and a hard rule can escalate to a block independently.

4.1.4 Feature Enrichment and the Trust Boundary

The client interface collects six fields, of which three reach the model. The model nevertheless receives thirteen features, assembled from three sources with deliberately different trust properties.

Table 4.1: Thirteen-Dimensional Feature Vector Composition

Source	Features	Trust Property
User-entered (3)	Amount, transaction type, merchant category	Client-asserted; anomalous values raise rather than lower the score, so misreporting is self-defeating
Client telemetry (2)	Device type, network type	Captured automatically at page load without user interaction; spoofing produces rarer, higher-scoring contexts
Server enrichment (8)	Sender and receiver age group, sender state, sender and receiver bank, hour of day, day of week, weekend flag	Resolved from the account database and the switch clock; not assertable by the client under any circumstances

This partition is a security design decision rather than an implementation convenience. The features that would most benefit an attacker if forged, namely a plausible transaction hour and a normal demographic profile, are exactly those resolved server-side. Enrichment imposes negligible latency because both account rows are already materialised for authentication and routing, so no additional query is issued.

4.2 Architecture Diagram

The architecture of the proposed system is presented in Figure 4.1. Three services cooperate over HTTP: a React client, a Spring Boot enterprise switch owning the relational ledger, and a FastAPI intelligence service hosting the hybrid scoring engine. The offline training pipeline, which produces the serialised model bundle, is shown separately because it never participates in the live payment path.

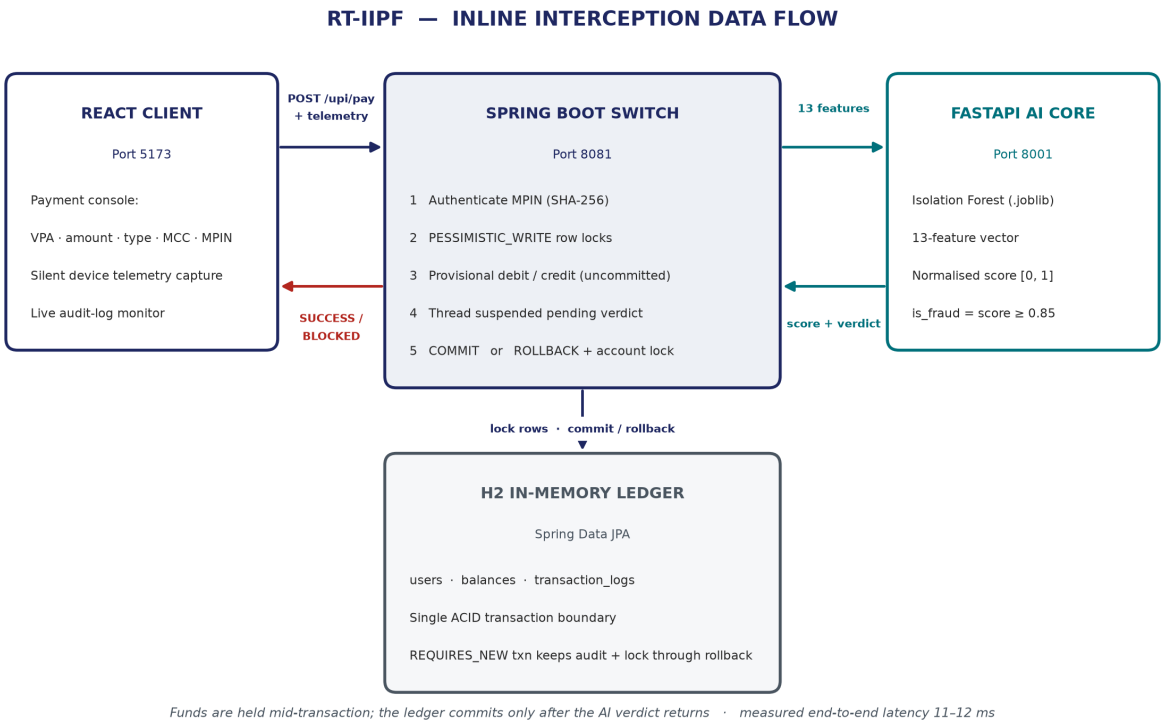


Fig 4.1: System Architecture of the Proposed RT-IIPF Framework

The numbered flow proceeds as follows. The client posts the payment together with the captured telemetry. The switch authenticates the MPIN, resolves both parties, and rejects the request if the payer account is already locked. It then acquires the two row locks, applies the provisional debit and credit, assembles the feature vector and issues a synchronous request to the intelligence service, at which point the payment thread is suspended. The service returns a fused score and verdict. A clean verdict causes the success audit record to be written and the transaction to commit; a fraudulent verdict causes the account lock and blocked audit record to be committed independently before the transaction is rolled back atomically. The client receives either a success response or a security-block response and renders the corresponding outcome.

4.3 Module Description and Workflow

The system comprises six modules. The first three constitute the live payment path, the fourth provides the intelligence, the fifth performs the offline preparation of the model, and the sixth provides evaluation.

4.3.1 Client Application Module

This module presents the payment console through which the user enters the payee virtual payment address, the amount, the transaction type and merchant category,

and the MPIN. Concurrently and without any user interaction, a telemetry component records the device class and network type reported by the browser, together with a derived device signature and a coarse location hash. These are transmitted with the payment in a single request. The module also renders the outcome, distinguishing a successful commit from a security block, and maintains a live operations view of recent decisions obtained by polling the switch.

4.3.2 Authentication and Validation Module

On receipt of a payment request the switch validates the payload against declared constraints, rejecting malformed requests before any database work is performed. It then resolves the payer and payee by virtual payment address. If the payer account carries a security lock the request is refused immediately. The MPIN supplied by the client is hashed and compared with the stored digest using a constant-time comparison, so that no information about the correct value leaks through response timing. Plaintext MPINs are never stored.

4.3.3 Ledger Interception Module

This module implements the core mechanism of the project. Within a single transactional boundary it acquires pessimistic write locks on the payer and payee balance rows, always in ascending account-identifier order so that two opposing concurrent payments cannot produce a circular wait. It verifies sufficient funds, applies the debit and credit to the persistence context without committing, and then suspends the executing thread while the intelligence service is consulted. On receiving the verdict it either writes the success audit record and commits, or delegates to the security module and raises the fraud exception that triggers atomic rollback.

4.3.4 Security Response Module

This module performs the writes that must survive the rollback. It executes in an independent transaction, which suspends the caller's transaction, sets the account lock flag on the payer record, writes the blocked audit record with its score and processing time, and commits. Only after this independent commit does control return to the interception module, which then raises the exception that discards the money movement. The design guarantees that a blocked attempt is permanently recorded even though the transfer it describes never existed in the committed ledger.

4.3.5 Hybrid Intelligence Module

The intelligence service loads the serialised bundle once at startup and performs a warm-up inference so that the first live request does not pay one-time initialisation costs. Each scoring request is validated against a declared schema, transformed by the preprocessor fitted during training, and evaluated by all three tiers. The rule engine produces a deterministic risk contribution and a hard-escalation flag, the Isolation Forest and the Local Outlier Factor each produce a calibrated score, and the fusion step combines them into the final verdict. The response carries not only the fused score and the binary decision but also the individual contribution of each tier, so that every block is explainable after the fact. If the bundle is absent the service refuses to start, on the principle that a scoring node without a model must never accept traffic.

4.3.6 Offline Training Module

This module prepares the model bundle and never participates in the live path. It ingests the transaction corpus, removes identifiers and any attribute knowable only after settlement so that no information can leak from the future into the feature vector, and partitions the data into training and held-out sets. Numeric attributes are standardised and categorical attributes one-hot encoded, with unseen categories handled gracefully so that an unfamiliar bank or state cannot cause a production failure. Both detectors are fitted, their calibration constants computed, the rule thresholds derived from the training distribution, and all artefacts serialised into a single bundle together with the fusion weights and the production threshold.

4.3.7 Evaluation Module and Experimental Results

The evaluation module supports the experimental work reported here. The public dataset carries a fraud annotation, but preliminary analysis showed it to be statistically uninformative: no detector evaluated, supervised or unsupervised, exceeded an ROC-AUC of 0.48 against it, a value consistent with random assignment. Reporting accuracy against such labels would be vacuous. The evaluation therefore adopts perturbation-based synthetic fraud injection, in which genuine held-out transactions are sampled and only the attributes defining each fraud archetype are overwritten, so that the injected vectors retain realistic marginal distributions and are not trivially separable.

Four archetypes were defined, one hundred instances each. Archetype A is an off-hour high-value account drain. Archetype B is a transaction-velocity attack and is

included deliberately as a negative control: its constituent transfers are individually unremarkable, and a scorer that examines each transaction independently is expected to detect none of them. Archetype C is a category anomaly, an implausibly large amount under an everyday merchant category. Archetype D is a context anomaly with a modest amount but an unusual hour and channel. Including the negative control quantifies a known architectural boundary rather than selecting archetypes favourable to the system.

Table 4.2 reports the performance of each tier and of the fused hybrid on the same evaluation set, with every score-based detector thresholded at an identical false-positive budget so that recall is compared at equal operational cost.

Table 4.2: Detector Performance at Matched False-Positive Rate

Detector	Precision	Recall	F1	FPR	ROC-AUC
Rule engine only	0.738	0.500	0.596	0.0014	—
Isolation Forest only	0.121	0.147	0.133	0.0086	0.723
Local Outlier Factor only	0.409	0.745	0.528	0.0086	0.896
Proposed hybrid engine	0.409	0.745	0.528	0.0086	0.881

The hybrid achieves a recall of 0.745 at a false-positive rate of 0.86 percent, a fivefold improvement over the Isolation Forest used alone. Two findings from this table merit emphasis because they are negative results obtained by measurement rather than assumption. First, the Isolation Forest is markedly the weakest constituent, which is attributable to its degradation on the sparse one-hot encoded categorical space and to score saturation beyond the training range. Second, the hybrid does not exceed the density-based tier used alone. Analysis of the per-archetype outcomes explains why: of the four hundred injected vectors, one hundred belong to the velocity negative control which no per-transaction detector can identify, and the hybrid detects two hundred and ninety-eight of the three hundred that are detectable at all, which is 99.3 percent of the achievable ceiling. There is therefore almost no headroom for any combination to exploit.

Table 4.3: Per-Archetype Detection Rate

Archetype	Isolation Forest	LOF	Proposed Hybrid
A – Off-hour high-value drain	0.26	1.00	1.00

B – Velocity attack (negative control)	0.00	0.01	0.01
C – Category anomaly	0.07	1.00	1.00
D – Context anomaly	0.26	0.97	0.97

The rule tier contributes no unique detections on this evaluation set, every transaction it flags being also flagged by the density tier. It is nevertheless retained for three reasons that recall alone does not capture: it attains a precision of 0.738 at a false-positive rate of only 0.14 percent, far more precise than either learned detector; it supplies a human-readable justification for a block, which an opaque anomaly score cannot; and it remains operative if the learned tiers are degraded or unavailable, providing a degraded-mode capability rather than outright rejection.

Latency was measured on the complete inline path and is reported in Table 4.4. A concurrency study was additionally performed at up to one hundred simultaneous clients. It established that the throughput ceiling of approximately ninety-five transactions per second is not caused by the pessimistic ledger locks, since contended and uncontended workloads were statistically indistinguishable, but by the single-process intelligence service. Because that service is stateless, replicating it across four worker processes raised throughput by a factor of between 3.4 and 5.4. The transactional mechanism and the scoring capacity therefore scale independently.

Table 4.4: Measured System Latency

Measurement	Value
End-to-end inline decision (single client, mean)	11–12 ms
End-to-end inline decision (99th percentile)	14 ms
Hybrid scoring hop including HTTP overhead	11–13 ms
Model bundle load at service startup (off the hot path)	≈ 1 s
Design budget	50 ms

4.4 Estimated Cost for Implementation and Overheads

The proposed system is implemented entirely with open-source software, and consequently the direct monetary cost of the Phase-I implementation is negligible. Table 4.5 itemises the components together with their licensing position and the overheads incurred.

Table 4.5: Estimated Cost and Implementation Overheads

S. No.	Component	Purpose	Cost

1	Java Development Kit 21	Switch runtime	Free (open source)
2	Spring Boot 3.5 with Spring Data JPA	Web and persistence framework	Free (open source)
3	H2 Database Engine	Embedded relational ledger	Free (open source)
4	Python 3 with FastAPI and Uvicorn	Intelligence service	Free (open source)
5	scikit-learn, pandas, NumPy, joblib	Model training and serialisation	Free (open source)
6	React 18 with Vite and Node.js	Client application	Free (open source)
7	UPI transaction dataset (public)	Model training corpus	Free
8	Development hardware	Standard laptop, no GPU required	Existing resource
		Total direct cost	Nil

The overheads that do arise are computational rather than financial. Training both detectors on two hundred thousand records completes in approximately one minute on a standard laptop. The serialised bundle occupies approximately sixteen megabytes and is loaded into memory once at startup, contributing about one second to service initialisation and nothing thereafter. At inference time the hybrid adds a few milliseconds per transaction relative to a single detector, which is accommodated comfortably within the design budget. Should the system be deployed at production scale, the principal recurring cost would be compute for the replicated stateless scoring tier, which the concurrency study indicates can be provisioned on commodity hardware in proportion to offered load.

REFERENCES

[1] Abdallah, A., Maarof, M. A., & Zainal, A. (2016). Fraud detection system: A survey. Journal of Network and Computer Applications, 68, 90–113.

[2] Adewumi, A. O., & Akinyelu, A. A. (2017). A survey of machine-learning and nature-inspired based credit card fraud detection techniques. International Journal of System Assurance Engineering and Management, 8(2), 937–953.

[3] Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. Proceedings of the ACM SIGMOD International Conference on Management of Data, 93–104.

[4] Campos, G. O., Zimek, A., Sander, J., Campello, R. J. G. B., Micenková, B., Schubert, E., Assent, I., & Houle, M. E. (2016). On the evaluation of unsupervised outlier detection: Measures, datasets, and an empirical study. Data Mining and Knowledge Discovery, 30(4), 891–927.

[5] Chandini, K., Mahender, A., & Venkateshwarlu, P. (2025). Detection of UPI fraudulent transactions through machine learning. International Journal of Engineering Research and Science & Technology, 21(4), 281–285.

[6] Deepak Kumar, A., Harshini Abirami, M., & Hepsi Nambika, J. (2026). Smart UPI fraud detect: ML-based anomaly identification in UPI payments. Proceedings of the 2026 International Conference on Emerging Technologies and Future Innovations (ETFI), 1–5.

[7] Fatlawi, W. B. (2025). Detecting better fraudulent detection using isolation forest and multi-cluster deep learning. Journal of Al-Qadisiyah for Computer Science and Mathematics, 17, 72–80.

[8] Hariri, S., Kind, M. C., & Brunner, R. J. (2021). Extended isolation forest. IEEE Transactions on Knowledge and Data Engineering, 33(4), 1479–1489.

[9] Jani, R., Kumar, M. R., Gumasthi, N., Singla, S., & Kailasam, T. (2025). Reinforcement learning for UPI fraud detection: Actor critic optimization approach. Artificial Intelligence Applications and Innovations (AIAI 2025). Springer.

[10] Kamble, V. B., Pisal, K., Vaidya, P., & Gaikwad, S. (2025). Enhancing UPI fraud detection: A machine learning approach using stacked generalization. International Journal of Multidisciplinary on Science and Management, 2(1), 69–83.

[11] Kavitha, J., Indira, G., Shrinita, A., & Bappan, D. (2024). Fraud detection in UPI transactions using ML. EPRA International Journal of Research and Development, 9(4), 142–146.

[12] Khopade, N. P., & Vitalkar, S. M. (2025). UPI fraud detection using machine learning. International Journal of Research in Interdisciplinary Studies, 3(6), 24–26.

[13] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 413–422.

[14] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. Neural Computation, 13(7), 1443–1471.

[15] Sindhu, J., & Swarupa, V. S. (2024). UPI fraud detection utilizing machine learning algorithms. International Journal of Engineering Research and Science & Technology, 20(3), 130–135.

[16] Thayanidhi, V., Srivarshan, M., & Shalini, M. (2026). UPI fraud detection using machine learning. Proceedings of the International Conference on Smart Data Intelligence (ICSMDI 2025). Springer.